Welcome

Object oriented programming

Logistics & Git Fundamentals

Instructor: Nadav Sudri (3rd Year, Ariel University)

Email: Nadavsu@ariel.ac.il

Phone: 050-944-5885

Assignments: 3-4 Home Assignments

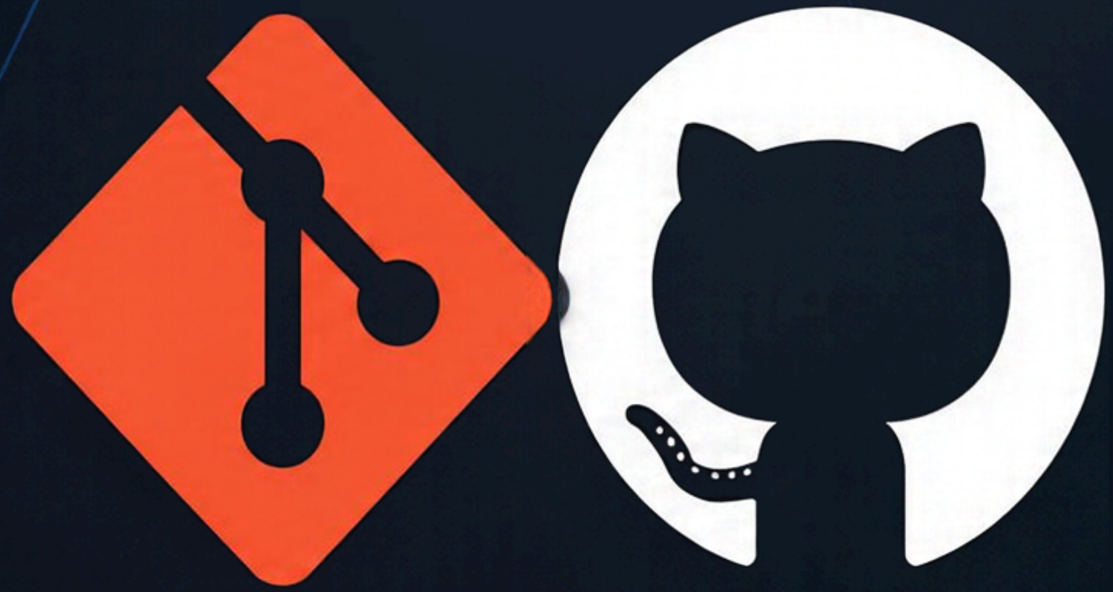
Final Grade: 1 Final Project (No exam!)

Office Hours : Via Email

Course matirials : Github



Part 1:
Version Control & GitHub



Git, GitHub, and Codespaces

https://www.youtube.com/watch?v=xvwBtODV0ms

Why Do We Need Git?

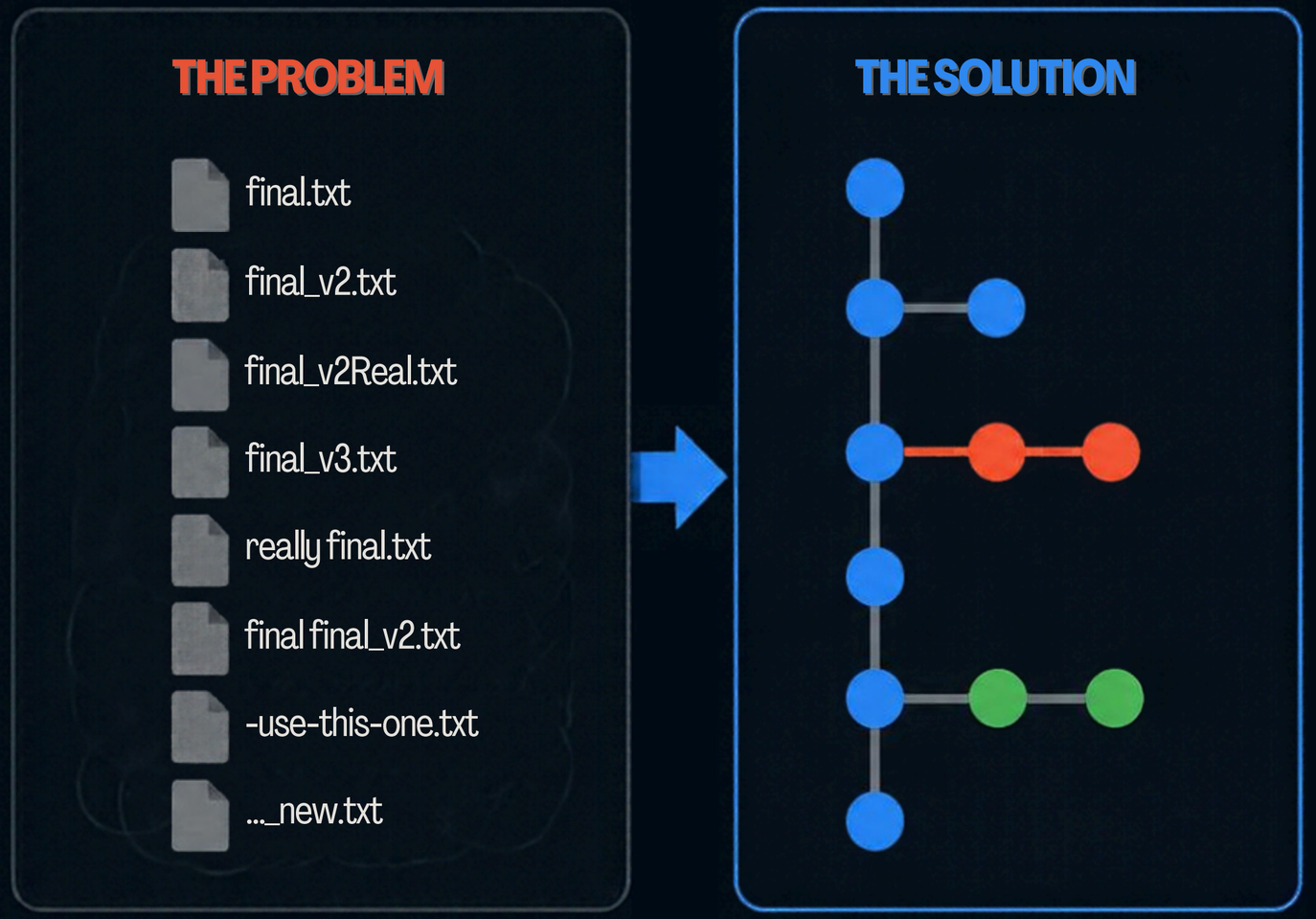
No more "final_v2 Real.txt":
Keeps a perfect history of your project.

Collaboration:
Work with multiple developers without overwriting each other's code.

Time Travel:
Made a mistake? Easily roll back to a previous working version.

Backup:

Your code is safely stored in the cloud (GitHub).



The Daily Developer Workflow

COMMAND	USE CASE	EXAMPLE
git clone	Download a project	git clone <url>
git status	Check what changed	git status
git add .	Stage all changes	git add main.py
git commit	Save a snapshot	git commit -m "fixed login bug"
git push	Upload to GitHub	git push origin main
git pull	Update local code	git pull

Logical Structure

Predictable folders make code easier to build, test, and hand off.

awesome-project/

```
├── src/
│   frontend/  api/  components/
├── tests/
│   unit/  integration/  e2e/
├── assets/
│   images/  fonts/  icons/
└── README.md
    setup + usage + contribution
```

Real-life scenarios

WEB APP
Frontend source, automated tests, and image assets stay easy to locate.

DATA SCIENCE
Notebooks, data, and trained models each have a clear home.

TEAM PROJECT
New contributors find the right file quickly—without guessing.

The README.md

Your project's front page: useful, scannable, and honest.

1 PURPOSE

Overview
What it does, who it is for, and project status.

2 SETUP

Installation
npm install
Usage
npm start
Configuration: list env vars and defaults.

3 CONFIDENCE

Tests: how to run checks.
Contributing: branches, reviews, coding rules.
License: name it clearly.

awesome-project

Markdown essentials

Heading
- Bullet
```bash
npm install
npm start
```

[Link](https://example.com) `inline code`

README ethics

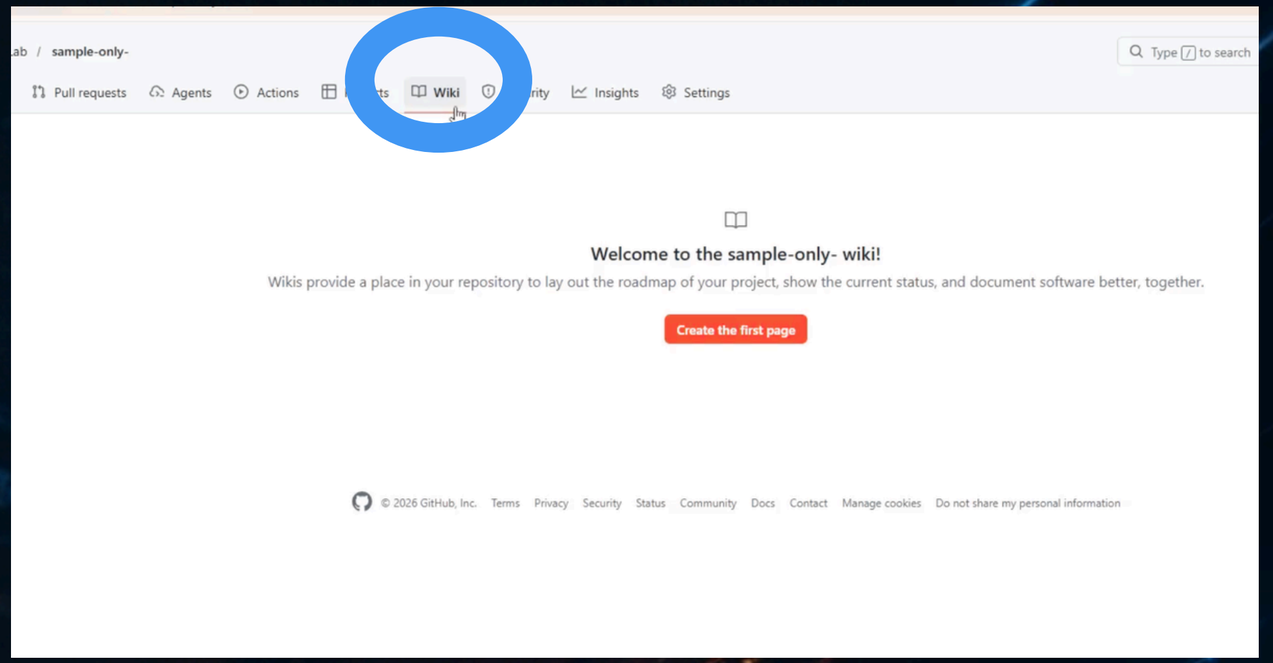
Be accurate and document prerequisites. Keep instructions current. Credit dependencies and contributors. Never publish secrets or private data.
Be honest about limitations.

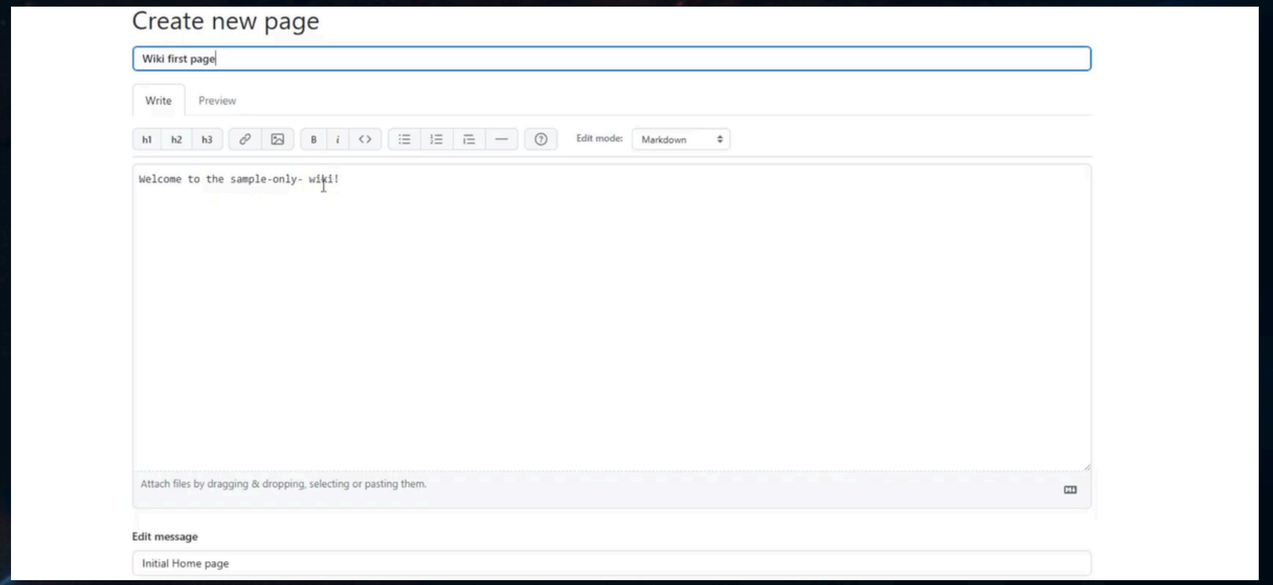
The Wiki

Why WIKI?

- The README is for the End-User: It answers, "What is this, how do I install it, and how do I run it?"
- The Wiki is for Developers and Contributors: It answers, "How does the architecture work under the hood? What are the API endpoints? How do I set up my local development environment to contribute?"

If multiple people are working on a project, the Wiki acts as the central source of truth for team operations. It holds coding standards, Git branching rules, meeting notes, and troubleshooting guides. It prevents the constant back-and-forth of "How do I configure this specific module?" because the answer is already documented.





GitHub Codespaces: Code in the Cloud

01 WHAT IS IT?

GitHub Codespaces is a cloud-hosted development environment for a GitHub repository.

Provides a ready-to-code Linux container with your project, dependencies, tools, extensions, and configuration.

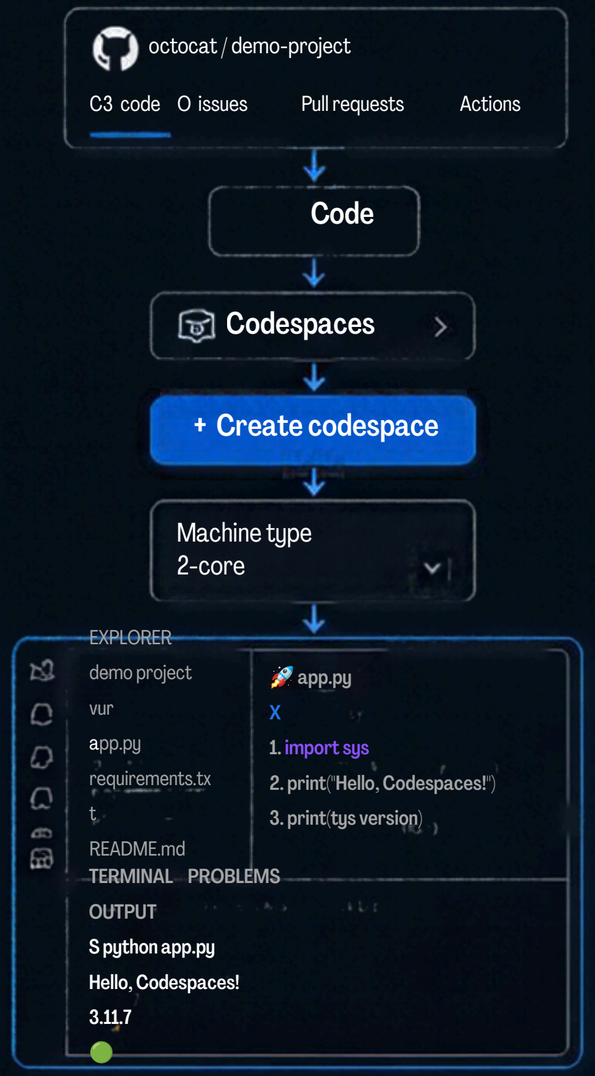
Code from anywhere on any device—no local setup required.

Perfect for class projects, pair programming, and team collaboration.

Open a repo. Start coding.

02 HOW TO USE IT

1. Open a GitHub repository
2. Code
3. Codespaces
4. Create codespace
6. Choose machine size if prompted
6. Edit, run, test, commit, and push



03 TOOLS + COST

TOOLS

- Browser-based VS Code
- VS Code desktop connection
- Integrated Terminal
- Debugger
- Extensions Marketplace
- Git built-in
- Port forwarding
- decontainer.json configuration
- GitHub CLI
- **JupiterLab** (optional)

COST

Free personal account:
120 core-hours/month
+ 15 GB-month storage

GitHub Pro:
180 core-hours/month +
20 GB-month storage

Paid usage after included quota:
compute while running + storage while the codespace exists. Stop unused codespaces and set a spending limit.

HANDS-ON PROJECT

Build, Commit, and Share

Choose a simple or semi-large code project, then take it from idea to public repository.

01 CREATE
Choose + build your project

02 COMMIT
Initialize Git + commit

03 PUBLISH
Create a repo + push

YOUR PROJECT

Choose a project idea
Define the goal and core features

Build a working first version
Add tests and clear documentation

PLAN • BUILD • TEST

Improve in small steps
Commit meaningful changes

Review your work
Prepare it for sharing



Ship it to GitHub

1 Initialize Git
```
git init
```

2 Commit your work
```
git add .
git commit -m "Initial commit"
```

3 Create a GitHub repository and copy its URL

4 Connect and push
```
git remote add origin <repo-url>
git branch -M main
git push -u origin main
```

Stretch goal
Extend the project, refine the README, and keep future changes committed.